01. Monad 블록체인 아키텍처 개요

개요

이 문서는 Monad 블록체인 실행 엔진의 전체 아키텍처를 설명합니다. Monad는 초고속 블록체인을 구현하기 위해 실행(Execution)과 합의(Consensus) 컴포넌트를 분리하고, 각 컴포넌트 내부에서 극한의 최적화 기법을 적용한 시스템입니다.

핵심 설계 철학

1. **컴포넌트 분리**: 실행과 합의를 독립적인 프로세스로 분리하여 각각 최적화
2. **병렬 처리**: 트랜잭션 병렬 실행, 비동기 I/O, 백그라운드 컴파일 등 모든 레벨에서 병렬화
3. **네이티브 성능**: EVM 바이트코드를 x86 머신 코드로 JIT 컴파일하여 네이티브 성능 달성
4. **최적화된 데이터베이스**: io_uring 기반 비동기 I/O와 효율적인 Merkle Patricia Trie 구현
5. **하드웨어 활용**: AVX2/AVX-512 SIMD 명령어, NVMe 직접 접근 등 최신 하드웨어 기능 활용

시스템 아키텍처

1. 실행(Execution)과 합의(Consensus) 분리

Monad는 두 개의 독립적인 데몬 프로세스로 구성됩니다:

```
┌──────────────────────────────────────────────────────────────────────┐
│ Consensus Daemon │
│ (monad-node binary) │
│ ┌──────────────────────────────────────────────────────────────┐ │
```

```
| | Monad BFT Consensus Algorithm | |
| | - 블록 제안 및 검증 | |
| | - 네트워크 통신 및 P2P 프로토콜 | |
| | - 노드 신원 관리 (공개키/개인키) | |
| | - 투표 및 합의 프로토콜 | |
| └──────────────────────────────────────────────┘ |
| ┌──────────────────────────────────────────────┐ |
| | libmonad_execution.so (Hosted Mode) | |
| | - TrieDB 읽기 API (상태 조회) | |
| | - RPC 서버용 트랜잭션 실행 (eth_call) | |
| └──────────────────────────────────────────────┘ |
└──────────────────────────────────────────────────┘
| writes blocks
▼
┌──────────────────────┐
| Ledger Directory |
| (Filesystem Storage) |
| - 최근 블록 저장 |
| - 합의가 쓰고 실행이 |
| 읽는 공유 저장소 |
└──────────────────────┘
| reads blocks
▼
```

```
┌──────────────────────────────────────────────────────────────┐
| Execution Daemon |
| (monad binary) |
| ┌──────────────────────────────────────────────────────┐ |
| | runloop_monad | |
| | - 블록 읽기 및 실행 루프 | |
| | - 새 블록 감지 및 자동 실행 | |
| └──────────────────────────────────────────────────────┘ |
| ┌──────────────────────────────────────────────────────┐ |
| | libmonad_execution.a (Static Library) | |
| | ┌──────────────────────────────────────────────┐ | |
| | | VM (category/vm) | | |
| | | - EVM 인터프리터 | | |
| | | - 네이티브 코드 컴파일러 (asmjit/LLVM) | | |
| | | - 컴파일된 코드 캐시 | | |
| | └──────────────────────────────────────────────┘ | |
| | ┌──────────────────────────────────────────────┐ | |
| | | Execution (category/execution) | | |
| | | - 트랜잭션 스케줄링 및 병렬 실행 | | |
| | | - 상태 관리 및 버전 제어 | | |
| | | - Precompile 구현 | | |
| | └──────────────────────────────────────────────┘ | |
| | ┌──────────────────────────────────────────────┐ | |
```

```
| | | TrieDB (category/mpt) | | |
| | | - Merkle Patricia Trie 구현 | | |
| | | - 상태 데이터베이스 (계정, 스토리지) | | |
| | | - 버전 관리 및 압축 | | |
| | └──────────────────────────────────────────┘ | |
| | ┌──────────────────────────────────────────┐ | |
| | | Async I/O (category/async) | | |
| | | - io_uring 통합 | | |
| | | - 비동기 디스크 I/O | | |
| | | - 버퍼 풀 관리 | | |
| | └──────────────────────────────────────────┘ | |
| └──────────────────────────────────────────────────┘ |
└──────────────────────────────────────────────────────────┘
| writes state
▼
┌──────────────────────┐
| TrieDB Database |
| (NVMe Block Device) |
| - 블록체인 상태 저장 |
| - 직접 I/O 최적화 |
└──────────────────────┘
```

2. 데이터 흐름

```
[합의 데몬] → [Ledger 디렉토리] → [실행 데몬] → [TrieDB]
↑ ↓
└──────────────── 상태 루트 검증 ────────────────┘
```

실행 흐름:

1. **블록 제안**: 합의 데몬이 새 블록을 제안하고 네트워크에서 투표
2. **블록 저장**: 합의된 블록을 Ledger 디렉토리에 기록
3. **블록 감지**: 실행 데몬이 새 블록을 감지
4. **트랜잭션 실행**: 실행 데몬이 블록 내 트랜잭션을 병렬로 실행
5. **상태 업데이트**: 실행 결과를 TrieDB에 기록
6. **상태 루트 계산**: Merkle 루트를 계산하여 합의 데몬에 전달
7. **검증**: 합의 데몬이 상태 루트를 검증

3. 라이브러리 사용 모드

`libmonad_execution` 은 세 가지 모드로 사용됩니다:

3.1 Stand-alone/Daemon Mode

- **바이너리**: `cmd/monad`
- **용도**: systemd 서비스로 실행
- **기능**: 블록 자동 실행 및 상태 업데이트
- **라이브러리**: `libmonad_execution.a` (정적 링크)

3.2 Hosted Mode

- **바이너리**: `monad-node` (합의 데몬)

- **용도**: 합의 데몬 내부에서 상태 조회
- **기능**: TrieDB 읽기 API, RPC 서버용 트랜잭션 실행
- **라이브러리**: `libmonad_execution.so` (동적 링크)

3.3 Interactive Mode

- **바이너리**: `cmd/monad` (CLI)
- **용도**: 개발 및 디버깅
- **기능**: 수동 블록 재생, 이더리움 메인넷 히스토리 검증

주요 컴포넌트 (category/)

Monad 실행 엔진은 8개의 주요 컴포넌트로 구성됩니다. 각 컴포넌트는 독립적인 디렉토리에 위치하며, CMake 서브프로젝트로 빌드됩니다.

1. async - 비동기 I/O 라이브러리

위치: `category/async/`

역할: io_uring 기반 비동기 I/O 인터페이스 제공

주요 파일:

- `io.hpp/cpp` : `AsyncIO` 클래스, io_uring 래퍼
- `io_senders.hpp` : Sender/Receiver 패턴 구현
- `storage_pool.hpp/cpp` : 청크 기반 스토리지 풀 관리
- `connected_operation.hpp` : 비동기 작업 체인

핵심 기능:

- io_uring SQE/CQE 처리
- 버퍼 풀 관리 (DMA 정렬)

- 비동기 읽기/쓰기 요청 제출
- 폴링 모드 지원

의존성:

- Boost.Fiber (비동기 작업 스케줄링)
- Linux io_uring (커널 5.1+)

CMake 타겟: `monad_async`

2. core - 공통 유틸리티

위치: `category/core/`

역할: 모든 컴포넌트가 공유하는 기본 기능 제공

주요 파일:

- `hash.hpp` : 해시 함수 (Keccak, BLAKE3)
- `keccak.hpp/c` : Keccak-256 구현
- `bytes.hpp` : 바이트 배열 타입
- `result.hpp` : Result<T, E> 타입 (에러 처리)
- `fiber/priority_pool.hpp/cpp` : 우선순위 기반 Fiber 풀
- `io/ring.hpp/cpp` : io_uring 저수준 래퍼
- `io/buffer_pool.hpp/cpp` : 버퍼 풀 구현
- `lru/static_lru_cache.hpp` : LRU 캐시

핵심 기능:

- 암호화 해시 함수
- 메모리 관리 (huge pages, 버퍼 풀)
- 동시성 프리미티브 (spinlock, fiber)

- 에러 처리 및 로깅
- 어셈블리 최적화 (x86-64)

의존성:

- Boost.Fiber, Boost.Stacktrace
- TBB (Threading Building Blocks)
- BLAKE3, ethash (Keccak)
- intx (256비트 정수)
- quill (로깅)

CMake 타겟: `monad_core`

3. execution - 트랜잭션 실행 및 스케줄링

위치: `category/execution/`

역할: 고수준 트랜잭션 처리 로직 및 이더리움 호환성

하위 디렉토리:

3.1 ethereum/ - 이더리움 호환 구현

주요 파일:

- `execute_block.hpp/cpp` : `execute_block_transactions()` - 블록 실행 메인 함수
- `execute_transaction.hpp/cpp` : `ExecuteTransaction` 클래스 - 단일 트랜잭션 실행
- `dispatch_transaction.hpp/cpp` : 트랜잭션 타입별 디스패칭
- `evmc_host.hpp/cpp` : EVMC 호스트 인터페이스 구현
- `precompiles.hpp/cpp` : 표준 precompile (ecrecover, sha256, ripemd160 등)
- `precompiles_bls12.hpp/cpp` : BLS12-381 precompile
- `state3/state.hpp/cpp` : `State` 클래스 - 상태 관리 및 버전 제어

- `chain/chain.hpp/cpp` : 체인 설정 (메인넷, 테스트넷)
- `core/` : 핵심 타입 (Address, Account, Transaction, Receipt, Block)
- `rlp/` : RLP 인코딩/디코딩
- `trace/` : 트랜잭션 추적 (call tracer, prestate tracer)

핵심 기능:

- 병렬 트랜잭션 실행 (Boost.Fiber 기반)
- 상태 충돌 감지 및 재실행
- 버전 스택을 통한 롤백 지원
- Precompile 구현
- 트랜잭션 검증 및 가스 계산

3.2 monad/ - Monad 전용 확장

주요 파일:

- `execute_system_transaction.hpp/cpp` : 시스템 트랜잭션 실행
- `monad_precompiles.hpp/cpp` : Monad 커스텀 precompile
- `staking/` : 스테이킹 메커니즘
- `staking_contract.hpp/cpp` : 스테이킹 컨트랙트 로직
- `read_valset.hpp/cpp` : 검증자 세트 조회
- `util/bls.hpp/cpp` : BLS 서명 검증
- `chain/` : Monad 체인 설정 (메인넷, 테스트넷, 데브넷)

의존성:

- monad-vm (EVM 실행)
- monad_trie (상태 데이터베이스)

- Boost.Fiber (병렬 실행)
- blst (BLS 서명)
- c-kzg-4844 (KZG 커밋먼트)

CMake 타겟: `monad_execution_ethereum`, `monad_execution_native`

4. mpt - Merkle Patricia Trie 데이터베이스

위치: `category/mpt/`

역할: 블록체인 상태를 저장하는 TrieDB (MonadDB) 구현

주요 파일:

- `db.hpp/cpp`: `Db` 클래스 - 데이터베이스 메인 인터페이스
- `trie.hpp/cpp`: Trie 구조 및 `UpdateAuxImpl` 클래스
- `node.hpp/cpp`: `Node` 클래스 - Trie 노드 (Empty, Leaf, Extension, Branch)
- `find.cpp`: 동기 조회 (`find()`, `get()`)
- `find_notify_fiber.cpp`: 비동기 조회
- `compute.hpp/cpp`: Merkle 루트 계산
- `update.hpp`: 업데이트 로직 (`upsert()`)
- `node_cache.hpp`: LRU 노드 캐시
- `nibbles_view.hpp`: Nibbles (4비트) 인코딩

핵심 기능:

- Merkle Patricia Trie 구현
- 버전 관리 (블록별 독립적인 루트)
- 압축(Compaction) - 오래된 버전 자동 정리
- 노드 캐싱 (가중치 기반 LRU)

- 비동기 I/O (io_uring 통합)
- 청크 기반 스토리지 (4MB 청크)

데이터베이스 모드:

- **RW (Read-Write)**: 읽기/쓰기 모드
- **RO (Read-Only)**: 읽기 전용 모드 (합의 데몬용)
- **InMemory**: 메모리 전용 모드 (테스트용)

의존성:

- monad_async (비동기 I/O)
- Boost.Fiber (비동기 작업)
- concurrentqueue (작업 큐)

CMake 타겟: `monad_trie`

CLI 도구: `monad_mpt` - TrieDB 관리 도구

5. vm - Ethereum Virtual Machine

위치: `category/vm/`

역할: EVM 바이트코드 실행 (인터프리터 + JIT 컴파일러)

하위 디렉토리:

5.1 interpreter/ - EVM 인터프리터

- `execute.cpp` : 인터프리터 실행 루프
- `intercode.cpp` : 중간 표현(Intercode) 생성
- `instruction_table.hpp` : 명령어 테이블
- `stack.hpp` : 스택 관리

5.2 compiler/ - 네이티브 코드 컴파일러

- `ir/x86.hpp` : x86 코드 생성
- `types.hpp` : 컴파일러 타입 정의

5.3 runtime/ - 런타임 지원 라이브러리

- `allocator.cpp` : 메모리 할당자 (`EvmStackAllocator` , `EvmMemoryAllocator`)
- `storage.cpp` : 스토리지 접근
- `math.S` : 어셈블리 최적화 수학 연산 (256비트 정수)
- `uint256.cpp` : 256비트 정수 구현
- `keccak.hpp` : Keccak 해시
- `context.S` , `exit.S` , `transmute.S` : 어셈블리 유틸리티

5.4 evm/ - EVM 명령어 정의

- `opcodes.hpp` : 옵코드 정의
- `traits.hpp` : EVM 버전별 특성 (Frontier, Homestead, ... Cancun)

주요 파일 (루트):

- `vm.hpp/cpp` : `VM` 클래스 - 실행 진입점
- `compiler.hpp/cpp` : `Compiler` 클래스 - 컴파일러 관리
- `varcode_cache.hpp/cpp` : `VarcodeCache` 클래스 - 컴파일된 코드 캐시

핵심 기능:

- EVM 바이트코드 인터프리터
- JIT 컴파일 (asmjit 사용)
- LLVM 백엔드 (옵션)
- 비동기 백그라운드 컴파일
- 코드 캐싱 (LRU)

- 어셈블리 최적화 (AVX2/AVX-512)

컴파일 프로세스:

```
EVM Bytecode → Intercode (중간 표현) → asmjit → x86 Machine Code → Cache
```

의존성:

- asmjit (JIT 어셈블러)
- LLVM (옵션, 고급 최적화)
- TBB (동시성)
- intx (256비트 정수)
- evmc (EVM C API)

CMake 타겟:

- `monad-vm` (메인)
- `monad-vm-interpreter`
- `monad-vm-compiler`
- `monad-vm-runtime`
- `monad-vm-evm`

6. rpc - RPC 인터페이스

위치: `category/rpc/`

역할: `extern "C"` 인터페이스로 단일 트랜잭션 실행 (RPC 서버용)

핵심 기능:

- `eth_call` 구현
- 읽기 전용 트랜잭션 실행

- 합의 데몬의 RPC 서버에서 사용

CMake 타겟: `monad_rpc`

7. statesync - 상태 동기화 프로토콜

위치: `category/statesync/`

역할: 노드 간 데이터베이스 상태 동기화

핵심 기능:

- 스냅샷 생성 및 전송
- 증분 동기화
- 압축 및 델타 전송
- `extern "C"` API (합의 데몬과 통신)

의존성:

- monad_trie (TrieDB 접근)
- 네트워크 프로토콜 (합의 데몬 제공)

CMake 타겟: `monad_statesync`

8. event - 실행 이벤트 SDK

위치: `category/event/` (실제 코드는 `category/core/event/`)

역할: 실시간 블록체인 데이터 스트리밍 (서드파티용)

핵심 기능:

- 이벤트 레코더
- 이벤트 이터레이터
- 링 버퍼 기반 이벤트 큐

CMake 타겟: `libmonad_event` (별도 라이브러리)

빌드 시스템 (CMake)

1. 루트 CMakeLists.txt 구조

위치: `CMakeLists.txt`

주요 설정:

```
cmake_minimum_required(VERSION 3.27)
project(monad)

# 컴파일러 요구사항
# GCC 15+ 또는 Clang 19+
if(CMAKE_CXX_COMPILER_ID STREQUAL "GNU")
if(CMAKE_CXX_COMPILER_VERSION VERSION_LESS 15)
message(FATAL_ERROR "GCC version 15 or higher is required.")
endif()
elseif(CMAKE_CXX_COMPILER_ID STREQUAL "Clang")
if(CMAKE_CXX_COMPILER_VERSION VERSION_LESS 19)
message(FATAL_ERROR "Clang version 19 or higher is required.")
endif()
endif()

# C++23 표준
set_property(TARGET ${target} PROPERTY CXX_STANDARD 23)
```

```
set_property(TARGET ${target} PROPERTY CXX_STANDARD_REQUIRED ON)
```

빌드 옵션:

```
option(MONAD_COMPILER_LLVM "Build llvm backend" OFF)

option(MONAD_COMPILER_COVERAGE "Build with coverage" OFF)

option(MONAD_COMPILER_TESTING "Build compiler tests" OFF)

option(MONAD_COMPILER_BENCHMARKS "Build compiler benchmarks" OFF)

option(MONAD_COMPILER_DUMP_ASM "Dump assembly files into build/asm" OFF)

option(MONAD_COMPILER_STATS "Print statistics about the compiler" OFF)

option(BUILD_SHARED_LIBS "Build monad with shared libraries" OFF)
```

컴파일 플래그 함수:

```
function(monad_compile_options target)

# C++23 표준

set_property(TARGET ${target} PROPERTY CXX_STANDARD 23)

set_property(TARGET ${target} PROPERTY CXX_STANDARD_REQUIRED ON)

# 경고 및 에러

target_compile_options(${target} PRIVATE -Wall -Wextra -Wconversion -Werror)

# GNU 소스 정의

target_compile_definitions(${target} PUBLIC "_GNU_SOURCE")

# Quill 로거 설정

target_compile_definitions(${target} PUBLIC QUILL_ROOT_LOGGER_ONLY)
```

```
endfunction()
```

2. 의존성 관리

2.1 Self-managed 의존성 (Git Submodules)

위치: `third_party/`

```
# asmjit - JIT 어셈블러
add_subdirectory(third_party/asmjit)

# BLAKE3 - 해시 함수
add_subdirectory(third_party/BLAKE3/c)

# blst - BLS 서명
include(cmake/blst.cmake)

# c-kzg-4844 - KZG 커밋먼트
add_subdirectory("third_party/c-kzg-4844-builder")

# ethash - Keccak 해시
add_subdirectory("third_party/ethash")

# intx - 256비트 정수
add_subdirectory("third_party/intx")
```

```
# evmc - EVM C API
add_subdirectory("third_party/evmc")

# nanobench - 벤치마크
add_subdirectory("third_party/nanobench")

# nlohmann_json - JSON 파싱
add_subdirectory("third_party/nlohmann_json")

# quill - 로깅
add_subdirectory("third_party/quill")

# silkpre - 서명 복구
add_subdirectory(third_party/silkpre)
```

2.2 System 의존성 (apt packages)

```
# Boost - 다양한 라이브러리
find_package(Boost REQUIRED COMPONENTS fiber json context stacktrace_basic)

# TBB - Threading Building Blocks
find_package(TBB REQUIRED)
```

```
# CLI11 - 커맨드 라인 파싱
find_package(CLI11 REQUIRED)


# GTest - 단위 테스트
find_package(GTest REQUIRED)


# PkgConfig 의존성
pkg_check_modules(brotli REQUIRED IMPORTED_TARGET libbrotlienc libbrotlidec)
pkg_check_modules(crypto++ REQUIRED IMPORTED_TARGET libcrypto++)
pkg_check_modules(archive REQUIRED IMPORTED_TARGET libarchive)
pkg_check_modules(zstd REQUIRED IMPORTED_TARGET libzstd)
```

Ubuntu 패키지 목록 (docker/Dockerfile 참조):

- `libboost-fiber-dev`, `libboost-json-dev`, `libboost-context-dev`
- `libtbb-dev`
- `liburing-dev` (io_uring)
- `libhugetlbfs-dev` (huge pages)
- `libbrotli-dev`, `libcrypto++-dev`
- `libarchive-dev`, `libzstd-dev`
- `libgtest-dev`

3. 컴포넌트 빌드 순서

```
# 1. 코어 라이브러리
```

```
add_subdirectory("category/core")


# 2. 비동기 I/O
add_subdirectory("category/async")


# 3. TrieDB (async 의존)
add_subdirectory("category/mpt")


# 4. VM (core 의존)
add_subdirectory("category/vm")


# 5. 실행 (vm, mpt 의존)
add_subdirectory("category/execution")


# 6. RPC, StateSync, Event
add_subdirectory("category/rpc")
add_subdirectory("category/statesync")
add_subdirectory("category/event")


# 7. 통합 라이브러리
add_library(monad_execution)
target_link_libraries(
monad_execution
```

```
PUBLIC monad_core
monad_async
monad_trie
monad_execution_ethereum
monad_execution_native
monad_rpc
monad_statesync
monad-vm
monad-vm-core
monad-vm-compiler
monad-vm-evm
monad-vm-interpreter
monad-vm-runtime
monad-vm-utils)


# 8. 실행 파일
add_subdirectory(${PROJECT_SOURCE_DIR}/cmd)
```

4. 빌드 명령어

설정 및 빌드:

```
# 환경 변수 설정
CC=gcc-15  CXX=g++-15  \
```

```
CFLAGS="-march=haswell" \
CXXFLAGS="-march=haswell" \
ASMFLAGS="-march=haswell" \
./scripts/configure.sh

# 빌드 (Ninja 사용)
./scripts/build.sh

# 또는 직접 빌드
cd build
ninja
```

빌드 타입:

- `RelWithDebInfo` (기본값): 최적화 + 디버그 심볼
- `Release` : 최대 최적화
- `Debug` : 디버그 전용

테스트 실행:

```
# 병렬 테스트
CTEST_PARALLEL_LEVEL=$(nproc) ctest

# 특정 테스트
ctest -R test_name
```

데이터 흐름 다이어그램

1. 블록 실행 흐름

```
[합의 데몬]
|
├─ 1. 블록 제안 및 투표
|
▼
[Ledger 디렉토리]
|
├─ 2. 블록 파일 기록
| (block_NNNNNN.bin)
|
▼
[실행 데몬 - runloop_monad]
|
├─ 3. 새 블록 감지
|
▼
[execute_block_transactions()]
|
├─ 4. 트랜잭션 서명 검증 (병렬)
```

```
| recover_senders()
|
├─ 5. 트랜잭션 병렬 실행
| ├─ Fiber 풀에 제출
| ├─ 낙관적 실행
| └─ 충돌 감지 및 재실행
|
▼
[ExecuteTransaction]
|
├─ 6. 상태 로드
| State::recent_account()
|
├─ 7. EVM 실행
| VM::execute()
| ├─ 인터프리터 또는
| └─ 컴파일된 네이티브 코드
|
├─ 8. 상태 업데이트
| State::set_storage()
|
▼
[TrieDB - Db::upsert()]
```

```
|
├─ 9. Trie 노드 업데이트
| UpdateAuxImpl::do_update()
|
├─ 10. 비동기 I/O
| AsyncIO::submit_write_request()
| io_uring
|
▼
[NVMe 디스크]
|
├─ 11. 상태 영구 저장
|
▼
[Merkle 루트 계산]
|
├─ 12. compute_root()
|
▼
[합의 데몬]
|
└─ 13. 상태 루트 검증
```

2. VM 실행 흐름

```
[EVM 바이트코드]
|
▼
[VM::execute()]
|
├─ 1. 코드 해시 계산
|
├─ 2. 캐시 조회
| VarcodeCache::get()
|
├─ 캐시 히트? ─┐
| |
▼ No ▼ Yes
[인터프리터] [네이티브 코드]
| |
├─ 3a. Intercode 생성 ├─ 3b. 직접 실행
| parser | execute_native_entrypoint_raw()
| |
├─ 4a. 인터프리터 실행 |
| execute_intercode_raw()
| |
```

```
├─ 5a. 백그라운드 컴파일 │
│ Compiler::async_compile()
│ (다음 실행을 위해) │
│ │
▼ ▼
[실행 결과 (evmc::Result)]
```

3. TrieDB 읽기 흐름

```
[상태 조회 요청]
│
▼
[Db::find(nibbles, block_id)]
│
├─ 1. 노드 캐시 확인
│ NodeCache::get()
│
├─ 캐시 히트? ─┐
│ │
▼ No ▼ Yes
[디스크 읽기] [캐시 반환]
│ │
├─ 2. 비동기 I/O 요청
```

```
| AsyncIO::submit_read_request()
|
├─ 3. io_uring SQE 제출
|
├─ 4. Fiber 대기
| (다른 작업 실행)
|
├─ 5. io_uring CQE 완료
| AsyncIO::poll_blocking()
|
├─ 6. 노드 역직렬화
| Node::deserialize()
|
├─ 7. 캐시 저장
| NodeCache::put()
|
▼
[노드 반환]
```

4. 병렬 실행 흐름

```
[블록 내 트랜잭션들]
|
```

```
▼
[execute_block_transactions()]
│
├─ 1. 서명 검증 (병렬)
│ ├─ Tx 0 ─┐
│ ├─ Tx 1 ─┤
│ ├─ Tx 2 ─┼─ recover_senders()
│ ├─ Tx 3 ─┤
│ └─ Tx 4 ─┘
│
├─ 2. Fiber 풀에 제출
│
▼
[Fiber Pool - 낙관적 실행]
│
├─ Tx 0 ──┐
├─ Tx 1 ──┤
├─ Tx 2 ──┼─ 동시 실행
├─ Tx 3 ──┤
└─ Tx 4 ──┘
│
├─ 3. 상태 접근 추적
│ ├─ Tx 0: 주소 A 읽기/쓰기
```

```
│ ├─ Tx 1: 주소 B 읽기/쓰기
│ ├─ Tx 2: 주소 A 읽기 ← 충돌!
│ ├─ Tx 3: 주소 C 읽기/쓰기
│ └─ Tx 4: 주소 B 쓰기 ← 충돌!
│
├─ 4. 충돌 감지
│ ├─ Tx 2는 Tx 0 이후 재실행 필요
│ └─ Tx 4는 Tx 1 이후 재실행 필요
│
▼
[재실행 (순차)]
│
├─ Tx 0 완료 → Tx 2 재실행
├─ Tx 1 완료 → Tx 4 재실행
│
▼
[모든 트랜잭션 완료]
│
└─ Receipt 수집 및 반환
```

핵심 최적화 기법 요약

1. 네이티브 코드 컴파일

- **기법**: EVM 바이트코드를 x86 머신 코드로 JIT 컴파일
- **도구**: asmjit (기본), LLVM (옵션)
- **효과**: 인터프리터 대비 10-100배 성능 향상
- **구현**: `category/vm/compiler/` , `category/vm/interpreter/`

2. 병렬 트랜잭션 실행

- **기법**: 낙관적 병렬 실행 + 충돌 감지 + 재실행
- **도구**: Boost.Fiber (경량 스레드)
- **효과**: 멀티코어 활용, 처리량 증가
- **구현**: `category/execution/ethereum/execute_block.cpp`

3. 비동기 I/O

- **기법**: io_uring 기반 비동기 디스크 I/O
- **도구**: Linux io_uring (커널 5.1+)
- **효과**: 디스크 레이턴시 숨김, 처리량 증가
- **구현**: `category/async/io.cpp`

4. 최적화된 데이터 구조

- **기법**: 효율적인 Merkle Patricia Trie, 노드 캐싱, 압축
- **도구**: LRU 캐시, 청크 기반 스토리지
- **효과**: 메모리 사용량 감소, 조회 속도 향상
- **구현**: `category/mpt/`

5. 어셈블리 최적화

- **기법**: 핵심 연산을 수작업 어셈블리로 구현
- **도구**: AVX2/AVX-512 SIMD 명령어
- **효과**: 256비트 정수 연산, 해시 함수 가속
- **구현**: `category/vm/runtime/math.S`, `category/core/keccak_impl.S`

6. 메모리 관리

- **기법**: 버퍼 풀, 캐시 재사용, Huge Pages
- **도구**: 커스텀 할당자, DMA 정렬
- **효과**: 할당 오버헤드 감소, 메모리 단편화 방지
- **구현**: `category/vm/runtime/allocator.cpp`, `category/core/io/buffer_pool.cpp`

하드웨어 요구사항

CPU 요구사항

최소 ISA: x86-64-v3 (Haswell 이상)

필수 명령어 세트:

- AVX2 (256비트 SIMD)
- BMI2 (Bit Manipulation Instructions)
- FMA (Fused Multiply-Add)
- MOVBE (Move Data After Swapping Bytes)

컴파일 플래그:

```
# 최소 요구사항
-march=x86-64-v3
```

```
# 또는
-march=haswell


# 로컬 실행 (최대 최적화)
-march=native
```

이유:

- AVX2: 256비트 정수 연산 가속 (Keccak, 수학 연산)
- BMI2: 비트 조작 최적화
- FMA: 부동소수점 연산 가속

메모리 요구사항

- **최소**: 32GB RAM
- **권장**: 64GB+ RAM
- **Huge Pages**: 2MB 페이지 지원 권장

스토리지 요구사항

- **타입**: NVMe SSD (필수)
- **용량**: 2TB+ (블록체인 상태)
- **IOPS**: 100K+ 읽기/쓰기
- **인터페이스**: PCIe 4.0 이상 권장

이유:

- io_uring은 NVMe의 병렬 I/O 큐를 최대한 활용
- 높은 IOPS로 상태 조회 레이턴시 최소화

컴포넌트 간 의존성 그래프

```
┌──────────────────────────────────────────────────────────────┐
| monad_execution |
| (통합 라이브러리) |
└──────────────────────────────────────────────────────────────┘
|
┌──────────────────┼──────────────────┐
| | |
▼ ▼ ▼
┌────────────────┐ ┌────────────────┐ ┌────────────────┐
| monad_core | | monad_async | | monad_trie |
| | | | | |
| - 공통 유틸 | | - io_uring | | - TrieDB |
| - 해시 함수 | | - 버퍼 풀 | | - MPT |
| - Fiber 풀 | | - Sender/ | | - 노드 캐시 |
| - 로깅 | | Receiver | | |
└────────────────┘ └────────────────┘ └────────────────┘
| | |
| └──────────┼──────────┘
| |
▼ ▼
┌────────────────┐ ┌────────────────┐
```

```
| monad-vm | | monad_ |
| | | execution_ |
| - VM | | ethereum |
| - Compiler | | |
| - Interpreter| | - 트랜잭션 |
| - Runtime | | 실행 |
| | | - 상태 관리 |
└──────────────┘ | - Precompile |
| └──────────────┘
| |
└──────────────┬──────────────┘
|
▼
┌──────────────┐
| monad_ |
| execution_ |
| native |
| |
| - Monad 확장 |
| - 스테이킹 |
| - 시스템 Tx |
└──────────────┘
|
```

```
┌──────────┬──────────┐
│ │ │
▼ ▼ ▼
┌─────────┐ ┌─────────┐ ┌─────────┐
│ monad_ │ │ monad_ │ │ monad_ │
│ rpc │ │ statesync│ │ event │
└─────────┘ └─────────┘ └─────────┘
```

의존성 설명:

1. **monad_core**: 모든 컴포넌트의 기반
2. **monad_async**: monad_trie가 의존 (비동기 I/O)
3. **monad_trie**: monad_execution이 의존 (상태 저장)
4. **monad-vm**: monad_execution이 의존 (EVM 실행)
5. **monad_execution_ethereum**: 이더리움 호환 로직
6. **monad_execution_native**: Monad 전용 확장
7. **monad_rpc, monad_statesync, monad_event**: 상위 레벨 서비스

주요 실행 파일

1. monad (실행 데몬)

위치: `cmd/monad/`

용도: 블록 실행 데몬

주요 기능:

- 블록 자동 실행 루프 (`runloop_monad`)
- 이더리움 메인넷 재생
- Monad 블록체인 실행
- 대화형 블록 실행

실행 예시:

```
# 데몬 모드
./build/cmd/monad --ledger-dir /path/to/ledger --db-path /path/to/db


# 이더리움 재생
./build/cmd/monad --chain ethereum --start-block 0 --end-block 1000000
```

2. monad_mpt (TrieDB 도구)

위치: `category/mpt/cli_tool_main.cpp`

용도: TrieDB 관리 및 디버깅

주요 기능:

- 데이터베이스 검사
- 노드 덤프
- 압축 실행
- 통계 출력

실행 예시:

```
# 데이터베이스 검사
```

```
./build/monad_mpt  inspect  --db-path  /path/to/db


# 특정 블록 상태 덤프

./build/monad_mpt  dump  --db-path  /path/to/db  --block-id  12345
```

3. benchmark_io_test (I/O 벤치마크)

위치: `category/async/test/benchmark_io_test.cpp`

용도: io_uring 성능 측정

주요 기능:

- 순차/랜덤 읽기/쓰기 벤치마크
- IOPS 측정
- 레이턴시 분석

테스트 구조

1. 단위 테스트

위치: `test/unit/`

프레임워크: Google Test

주요 테스트:

- `category/vm/test/` : VM 컴포넌트 테스트
- `category/mpt/test/` : TrieDB 테스트
- `category/async/test/` : 비동기 I/O 테스트
- `category/execution/ethereum/` : 실행 로직 테스트

- `category/execution/monad/` : Monad 확장 테스트

실행:

```
# 모든 테스트
ctest

# 특정 컴포넌트
ctest -R monad_vm
ctest -R monad_trie
```

2. 통합 테스트

위치: `test/ethereum_test/`

용도: 공식 이더리움 테스트 스위트 실행

테스트 종류:

- 블록체인 테스트
- 상태 전이 테스트
- VM 테스트
- 트랜잭션 테스트

데이터: `third_party/ethereum-tests/` (Git 서브모듈)

3. 벤치마크

위치:

- `category/vm/bench/` : VM 성능 벤치마크

- `category/mpt/bench/` : TrieDB 성능 벤치마크

프레임워크: nanobench

주요 벤치마크:

- BurntPix: 순수 계산 성능 (EVM 루프)
- 실제 트랜잭션 재생
- 디스크 I/O 성능

개발 워크플로우

1. 코드 체크아웃

```
# 저장소 클론
git clone https://github.com/category-labs/monad.git
cd monad

# 서브모듈 초기화
git submodule update --init --recursive
```

2. 의존성 설치 (Ubuntu)

```
# 시스템 패키지
sudo apt update
sudo apt install -y \
gcc-15 g++-15 \
```

```
cmake  ninja-build \
libboost-fiber-dev libboost-json-dev libboost-context-dev \
libtbb-dev \
liburing-dev libhugetlbfs-dev \
libbrotli-dev  libcrypto++-dev \
libarchive-dev libzstd-dev \
libgtest-dev
```

3. 빌드

```
# 설정 (RelWithDebInfo)
CC=gcc-15  CXX=g++-15  \
CFLAGS="-march=haswell" \
CXXFLAGS="-march=haswell" \
ASMFLAGS="-march=haswell" \
./scripts/configure.sh

# 빌드
./scripts/build.sh

# 또는 디버그 빌드
CMAKE_BUILD_TYPE=Debug  ./scripts/configure.sh
./scripts/build.sh
```

4. 테스트

```
# 모든 테스트 (병렬)
cd build
CTEST_PARALLEL_LEVEL=$(nproc) ctest

# 특정 테스트
ctest -R test_name -V
```

5. 코드 포맷팅

```
# clang-format 적용
./scripts/apply-clang-format.sh

# clang-tidy 검사
./scripts/check-clang-tidy.sh
```

디렉토리 구조 요약

```
monad/
├── category/ # 실행 엔진 소스 코드
| ├── async/ # 비동기 I/O (io_uring)
| ├── core/ # 공통 유틸리티
```

```
| ├── event/ # 이벤트 SDK
| ├── execution/ # 트랜잭션 실행
| | ├── ethereum/ # 이더리움 호환
| | └── monad/ # Monad 확장
| ├── mpt/ # TrieDB (Merkle Patricia Trie)
| ├── rpc/ # RPC 인터페이스
| ├── statesync/ # 상태 동기화
| └── vm/ # EVM 구현
| ├── compiler/ # JIT 컴파일러
| ├── interpreter/ # 인터프리터
| ├── runtime/ # 런타임 라이브러리
| ├── evm/ # EVM 명령어 정의
| └── llvm/ # LLVM 백엔드 (옵션)
├── cmd/ # 실행 파일
| ├── monad/ # 실행 데몬
| └── vm/ # VM 테스트 도구
├── cmake/ # CMake 설정
├── docker/ # Docker 빌드 파일
├── docs/ # 문서
├── scripts/ # 빌드 및 테스트 스크립트
├── test/ # 테스트
| ├── ethereum_test/ # 이더리움 테스트 스위트
| ├── unit/ # 단위 테스트
```

```
│ └── vm/ # VM 테스트
├── third_party/ # 서드파티 라이브러리 (서브모듈)
│ ├── asmjit/ # JIT 어셈블러
│ ├── BLAKE3/ # 해시 함수
│ ├── blst/ # BLS 서명
│ ├── ethash/ # Keccak 해시
│ ├── evmc/ # EVM C API
│ ├── intx/ # 256비트 정수
│ ├── nlohmann_json/ # JSON 파싱
│ ├── quill/ # 로깅
│ └── ...
├── CMakeLists.txt # 루트 CMake 설정
└── README.md # 프로젝트 개요
```

참고 자료

공식 문서

- Monad 공식 문서
- Monad 아키텍처
- 하드웨어 요구사항
- 네이티브 컴파일
- 병렬 실행

- MonadDB

저장소

- monad (실행)
- monad-bft (합의)

기술 스택

- asmjit - JIT 어셈블러
- io_uring - 비동기 I/O
- Boost.Fiber - 경량 스레드
- TBB - Threading Building Blocks
- EVMC - Ethereum VM C API

이더리움 표준

- Ethereum Yellow Paper
- EVM Opcodes
- Merkle Patricia Trie

다음 문서

이 아키텍처 개요를 바탕으로, 다음 문서들에서 각 컴포넌트의 세부 구현을 분석합니다:

- **02-VM-네이티브-컴파일.md**: EVM 인터프리터와 JIT 컴파일러 상세 분석
- **03-병렬-실행-스케줄링.md**: 트랜잭션 병렬 실행 메커니즘
- **04-TrieDB-상태-관리.md**: Merkle Patricia Trie 구현 및 최적화
- **05-비동기-IO-최적화.md**: io_uring 통합 및 버퍼 관리

- **06-런타임-최적화.md**: 어셈블리 최적화 및 메모리 관리
- **07-Monad-확장-기능.md**: Monad 전용 기능 (스테이킹, precompile 등)
- **08-상태-동기화.md**: 노드 간 상태 동기화 프로토콜
- **09-성능-분석.md**: 벤치마크 결과 및 최적화 효과
- **10-빌드-및-배포.md**: 빌드 시스템 및 배포 가이드